MLSecOps Robustness Gate

PyTorch 模型的自動化對抗式強健性檢測

專案介紹文件：檔案導覽＋程式原理思路

2026-10-03

1. 這是什麼

一句話：每次模型有更新，自動跑一輪「模擬駭客攻擊」，檢查模型擋不擋得住；擋不住，就擋下發布。

它解決的問題：機器學習模型上線前，幾乎沒人系統性地測試它怕不怕對抗式攻擊（adversarial attack）——一種加了人眼看不出的微小擾動、就能讓模型認錯的攻擊。傳統做法是人工偶爾測一次，這個專案把它變成 CI pipeline 裡的一個 gate：每次 push、PR 或每天半夜自動跑，PASS 才能合併，FAIL 直接擋下。

Workflow 六個階段

- 1. Load：載入待測模型、policy（門檻設定）、測試資料
- 2. Baseline：先量乾淨準確率（clean accuracy），當作比較基準
- 3. Attack Suite：5 種攻擊 × 3 種強度（eps），產生對抗樣本
- 4. Metrics：算 robust accuracy、ASR（攻擊成功率）
- 5. Policy Check：把指標拿去對 YAML 寫的門檻，一條一條檢查
- 6. Verdict：輸出 PASS / FAIL，寫成 report.md（人看）＋ report.json（CI 看）；FAIL 時程式回傳 exit 1，讓 CI 直接紅燈

2. 檔案導覽：每個重要檔案在做什麼

檔案	負責的事
robogate/attacks.py	5 種攻擊的 PyTorch 實作
robogate/data.py	合成測試資料集（免下載）
robogate/model_zoo.py	內建測試模型＋訓練函式
robogate/metrics.py	指標計算
robogate/policy.py	讀 YAML、判 PASS / FAIL
robogate/gate.py	六階段 workflow 指揮中心
robogate/report.py	產出 md / json 報告
examples/quickstart.py	一鍵 demo
examples/compare_models.py	三種架構 robustness 對比
policies/default.yaml	預設門檻
tests/test_logic.py	10 個純邏輯測試

3. 核心原理與思路

3.1 對抗式攻擊是什麼

直覺：一張圖片加上人眼看不出的微小噪聲，模型就認錯了。攻擊者不是亂加噪聲，而是用模型的 gradient 算出「最有效的噪聲方向」。威脅模型分兩種：white-box（看得到模型內部、能算 gradient）跟 black-box（只能呼叫模型、看回傳的分數）。擾動大小用 L_inf 限制：每個 pixel 最多改 eps。

3.2 FGSM：一步到位

思路：把 loss 對 input 微分，得到「讓 loss 上升最快的方向」，然後一次走 eps 那麼遠。

```
x_adv = x + eps * sign(gradient of loss w.r.t. x)
```

只走一步，所以最快、也最弱，適合當 smoke test。

3.3 PGD / BIM：切成很多小步

思路：FGSM 一次走到底容易走過頭，切成很多小步、每步走一點、再投影回 eps 球裡面，慢慢爬到 loss 最高的地方。PGD 多了一個隨機起點（random start），避免卡在局部陷阱；BIM 就是沒有隨機起點的版本。

```
x(t+1) = clip( x(t) + alpha * sign(grad), x - eps, x + eps )
```

迭代攻擊比單步強得多，是 white-box 檢驗的標準配備。實測裡 PGD 在 eps=0.1 就把 demo 模型的 robust accuracy 打到接近 0%。

3.4 NES：看不到 gradient 就用猜的（black-box）

思路：真實攻擊者通常沒有模型權重，只能呼叫 API 看分數。NES 在 x 附近撒一堆隨機噪聲點，只看模型回傳的分數，用「分數 × 噪聲方向」加權平均，估出 gradient 的方向：

```
estimated_grad = mean( loss(x + noise_i) * noise_i ) / sigma
```

每次撒噪聲時同時撒反方向（antithetic pairs）來降低估計的變異數。意義：它模擬的是真實世界最常見的威脅模型，代價是 query 次數多、比白盒弱很多——實測裡 NES 在 eps=0.1 幾乎打不動 CNN，只在 eps=0.2 打出 70% ASR。

3.5 Noise：對照組

純隨機均勻噪聲，完全不看模型。如果模型連隨機噪聲都怕，那問題不在攻擊者有多厲害，在模型本身太脆弱。實測裡 noise 的 ASR 是 0%，表示 demo 模型至少不是紙糊的。

3.6 指標怎麼算

· clean accuracy：在乾淨測資上的準確率，當作基準線。
· robust accuracy：被攻擊後的圖片還能答對的比例。越高越好。
· ASR（attack success rate）：原本答對、被攻擊後答錯的比例。越低越好。
· 關係：robust_acc ＝ clean_acc −（被翻轉的比例）。看 robust_acc 就知道模型還剩多少，ASR 告訴你攻擊有多有效。

3.7 Policy 與 verdict

policies/default.yaml 寫的是白紙黑字的門檻，例如「pgd 在 eps=0.1 時 robust_acc 不得低於 70%」。policy.py 的 check() 把實測數字一條一條拿去對，沒過就記一條 violation；只要有一條 violation，verdict 就是 FAIL。quickstart.py 在 FAIL 時讓程式以 exit 1 結束——CI 看到非零 exit code 就直接紅燈擋下 merge，不需要人盯著看報表。

4. 模型動物園：三種架構在比什麼

· TinyCNN：2 層卷積的小 CNN，demo 預設。重點是快，幾十秒訓完。

· MLP：全連接網路，沒有卷積。設計思路是當對照組——拿掉「局部性＋權重共享」的歸納偏置，看看這對 robustness 有沒有幫助。

· DeepCNN：3 層卷積。設計思路：更深有沒有更耐打？

下面是實際跑出來的對比（eps=0.1，數字是 robust accuracy，越高越好）：

model	clean	fgsm	pgd	bim	nes	noise
tinycnn	100%	62%	3%	4%	100%	100%
mlp	97%	0%	0%	0%	3%	93%
deepcnn	100%	23%	0%	0%	94%	100%

怎麼解讀：第一，MLP 在白盒攻擊下全滅（0%）——沒有空間結構假設，gradient 處處都可以攻擊，這正是卷積的歸納偏置帶來的紅利。第二，CNN 家族 FGSM 還能撐一下，但 PGD/BIM 照樣全倒——架構有差，但沒做 adversarial training 的模型，白盒就是擋不住。第三，NES 幾乎只打得動 MLP——black-box 攻擊成本高、效果差，這也是為什麼真實攻擊者更愛找白盒或半白盒的破口。

5. 資料集：為什麼用合成的

data.py 現場產生 16×16 灰階圖：class 0 是圓形 blob，class 1 是方形 blob，再加 20% 隨機噪聲。用合成資料的三個理由：零下載、離線可跑；deterministic，seed 固定結果可重現；任務夠簡單，小模型幾十秒就收斂，適合 demo 跟 CI。代價也要說清楚：結論不能直接推到真實任務上。正式用法是換成你自己的資料集跟模型（下一章）。

6. 實測報表怎麼看（以 VM 跑的那次為例）

clean accuracy 100%，表示模型正常任務滿分。但 pgd 在 eps=0.1 時 robust accuracy 只剩 0.5%（ASR 99.5%），幾乎全滅；fgsm 在 eps=0.1 剩 53%，也沒過 80% 的門檻。於是 violations 記了兩條，verdict FAIL。看報表的順序：先看 verdict，再看 violations 知道哪條門檻沒過，最後看表格找是哪個（攻擊、強度）組合最脆弱——那就是你要優先修的地方。

7. 換成自己的模型

Gate 只要求兩件事：模型是 torch.nn.Module（輸入 tensor、輸出 logits），測試資料是 DataLoader（x 值域 [0,1]，y 是標籤）。

```
model = torch.load("your_model.pt", weights_only=False)
model.eval()
gate = Gate(model, your_test_loader, ATTACKS,
        [0.05, 0.1, 0.2], load_policy("policies/default.yaml"))
results = gate.run()  # results["verdict"] 就是 PASS / FAIL
```

注意 eps 要配合 input 的尺度調：圖片 [0,1] 用 0.05～0.2 合理；別的資料型態自己定。policy 的門檻也要按你的場景重寫，default.yaml 只是 demo 用的。

8. 下一步（Roadmap）

- 更強的 black-box 攻擊：Boundary Attack、Square Attack
- Certified robustness：randomized smoothing，給出數學保證而不只是實測
- 接 MLflow model registry：模型一註冊自動觸發 gate
- 趨勢儀表板：追蹤每次 commit 的 robustness 變化